

Lab 7 — Sınıflar, Nesneler, Kapsülleme ve Soyutlama

YZG102 — C++ ile Nesne Yönelimli Programlama

Yesim Yigitbasi 07.05.2026

Gorev 1

Dikdortgen

Class (sınıf) yapısını kullanarak gerçek hayattan basit bir nesneyi modelleyin: Dikdörtgen.

Bir dikdörtgenin iki temel özelliği vardır:

- genişlik
- yükseklik

Ayrıca bu bilgilerden hesaplanabilen iki önemli değeri vardır:

- Alan = genişlik $\times$ yükseklik
- Çevre = $2 \times$ (genişlik + yükseklik)

Kod tamamlandığında program 3 farklı dikdörtgen oluşturacak ve her biri için:

- Boyut
- Alan
- Çevre

bilgilerini ekrana yazdırın.

class ile struct arasındaki tek sözdizimsel fark nedir?

Nesne ile sınıf arasındaki kavramsal fark (ev-ev planı metaforu):

class ile struct arasındaki tek sözdizimsel fark nedir?

Varsayılan erişim belirleyicisidir. struct içindeki üyeler varsayılan olarak public'tir, yani dışarıdan doğrudan erişilebilir. class içindeki üyeler ise varsayılan olarak private'tir, dışarıdan erişilemez. Her iki yapı da üye fonksiyon, kalıtım ve kapsülleme destekler. C++'da struct genellikle sade veri gruplamak için, class ise OOP uygulamak için kullanılır.

Nesne ile sınıf arasındaki kavramsal fark (ev-ev planı metaforu):

Sınıf bir ev mimarisi planına benzer: kaç oda olacağını, mutfağın nerede olacağını, evin hangi özelliklerine sahip olacağını tanımlar. Ama plan tek başına yaşanabilir bir ev değildir. Nesne ise bu plandan inşa edilmiş gerçek bir evdir. Birden fazla ev aynı plandan yapılabilir (birden fazla nesne aynı sınıftan üretilebilir) ve her evin kendine ait içi vardır (her nesne kendi verilerini bağımsız saklar).

Gorev 2

Sıcaklık Dönüştürücü

Kapsülleme (encapsulation) mantığını kullanarak bir Sıcaklık Dönüştürücü Sistemi geliştirin. Gerçek hayatta sıcaklık farklı birimlerle ifade edilir: Celsius (°C), Fahrenheit (°F), Kelvin (K)

Bu sınıfta temel veri olarak yalnızca **Celsius** değeri tutulacaktır. Diğer sıcaklık türleri bu değerden hesaplanacaktır. Bu görevde önemli nokta şudur: celsius değişkeni **private** olarak tanımlanmıştır. Yani dışarıdan doğrudan erişilemez.

Bunun yerine kontrollü erişim sağlamak için:

- setCelsius() → değer atamak için
 - getCelsius() → Celsius okumak için
 - getFahrenheit() → Fahrenheit hesaplamak için
 - getKelvin() → Kelvin hesaplamak için
- fonksiyonları kullanılacaktır.

Ayrıca sistem bir doğrulama da yapmaktadır:

- Mutlak sıfır = -273.15°C
- Bunun altındaki sıcaklıklar fiziksel olarak mümkün değildir.

Bu nedenle kullanıcı geçersiz bir değer girerse sistem hata mesajı verir ve eski değeri korur.

Kod tamamlandığında program:

- Girilen sıcaklıkları gösterecek
- Fahrenheit ve Kelvin karşılıklarını hesaplayacak
- Geçersiz girişleri engelleyecek

celsius deęiřkeni public olsaydı ne gibi sorunlar ıkabilirdi?

Bu sınıfta soyutlama (abstraction) nasıl uygulanmaktadır?

celsius değişkeni public olsaydı ne gibi sorunlar çıkabilirdi?

Dışarıdan `s1.celsius = -5000;` gibi fiziksel olarak imkânsız değerler atanabilirdi. Mutlak sıfır (-273.15°C) altında sıcaklık yoktur ama bunu engelleyecek bir mekanizma olmazdı. Aynı şekilde yanlışlıkla fahrenheit değeri celsius alanına yazılabilirdi. Kapsülleme sayesinde `setCelsius()` içindeki doğrulama her türlü geçersiz girişi engeller ve veri bütünlüğü korunur.

Bu sınıfta soyutlama (abstraction) nasıl uygulanmaktadır?

Kullanıcı, sıcaklığın içeride Celsius cinsinden saklandığını bilmek zorunda değildir. `getFahrenheit()` veya `getKelvin()` çağrısı yapıldığında dönüşüm formülleri tamamen gizlidir. Kullanıcı sadece `yazBilgi()` gibi sade bir arayüzle tüm birimleri görebilir. Dönüşümün nasıl yapıldığı, hangi formülün kullanıldığı sınıf içinde gizlenmiştir. Bu, karmaşıklığı kullanıcıdan gizleyip yalnızca gerekli arayüzü sunmak olan soyutlamanın tanımıyla örtüşür.

Gorev 3

Banka Hesabı

Getter / setter mantığı, kapsülleme ve nesne ile veri yönetimi konularını kullanarak basit bir Banka Hesabı Sistemi geliştirin.

Gerçek hayatta bir banka hesabında bazı bilgiler bulunur:

- Hesap sahibinin adı
- Hesap numarası
- Güncel bakiye
- Yapılan işlem sayısı

Bu bilgiler doğrudan herkes tarafından değiştirilmemelidir. Örneğin kullanıcı:

- Bakiyeyi rastgele eksi yapamamalı
- Hesaba doğrudan para ekleyememeli
- İşlem sayısını elle değiştirememeli

Bu nedenle tüm veriler private olarak saklanmıştır. Dışarıdan erişim yalnızca kontrollü fonksiyonlarla sağlanır:

Para işlemleri

- yatır(miktar) → hesaba para ekler
- çek(miktar) → yeterli bakiye varsa para çeker

Bilgi alma

- getBakiye() → mevcut bakiyeyi döndürür
- getIslemSayisi() → toplam işlem sayısını verir

Hesap oluşturma

- hesapAc(...) → yeni hesap bilgilerini tanımlar

Raporlama

- ekstreYazdir() → hesap özetini ekrana yazdırır

setter olmadan sadece getBakiye() olsaydı sorun olur muydu?

islemSayisi değişkenini neden public yapmak yerine getter ile sundunuz?

setter olmadan sadece getBakiye() olsaydı sorun olur muydu?

Hayır, sorun olmazdı çünkü getter yalnızca okuma yapar ve veriyi değiştirmez. Ancak bakiyeyi değiştirmek isteyen kod için setter (yatir, cek gibi fonksiyonlar) gereklidir. Getter'ın amacı private veriyi dışarıya güvenle sunmaktır. Setter'ın amacı ise veriyi doğrulayarak değiştirmektir. İkisi birbirinin yerini tutmaz.

islemSayisi değişkenini neden public yapmak yerine getter ile sundunuz?

islemSayisi değişkeni public olsaydı dışarıdan h1.islemSayisi = 999; gibi sahte bir değer atanabilirdi. Bu veri bütünlüğünü bozar. Getter olan getIslemSayisi() ile yalnızca okuma izni verilir, yazma izni verilmez. Böylece işlem sayısı yalnızca yatir() ve cek() fonksiyonları çağrıldığında artar. Kapsüllemenin özü budur: okuma ve yazma izinlerini ayrı ayrı kontrol etmek.

Gorev 4

Öğrenci Mevcudu

Aynı sınıftan birden fazla nesne oluşturarak basit bir öğrenci mevcudu sistemi geliştirin. Burada her öğrenci bir nesne olarak temsil edilir.

Her öğrencinin kendi bilgileri vardır:

- Ad
- Soyad
- Numara
- Vize notu
- Final notu
- Ortalama
- Harf notu

Bu bilgiler her öğrenci için ayrı ayrı saklanır. Yani bir öğrencinin notunu değiştirmek diğer öğrenciyi etkilemez.

Program 4 öğrenciden oluşan bir sınıf oluşturacak.

Her öğrenci için:

- Bilgiler girilecek
- Ortalama hesaplanacak
- Harf notu belirlenecek
- Bilgiler ekrana yazdırılacak

Daha sonra program:

1. Sınıf ortalamasını hesaplayacak
2. En yüksek ortalamaya sahip öğrenciyi bulacak
3. En iyi öğrencinin bilgilerini ekrana yazdıracak

sinif[0] nesnesinin verisini değiştirmek sınıf[1]'i etkiler mi? Neden?

getHarfNotu() fonksiyonu neden public olmalıdır?

sinif[0] nesnesinin verisini değiştirmek sınıf[1]'i etkiler mi? Neden?

Hayır, etkilemez. Aynı sınıftan üretilen her nesne bellekte tamamen bağımsız alanda tutulur. sınıf[0].vize ile sınıf[1].vize birbirinden farklı bellek adreslerindedir. Birini değiştirmek diğerini etkilemez. Bu, nesne yönelimli programlamanın temel özelliklerinden biridir: her nesne kendi durumunu (state) bağımsız olarak yönetir.

getHarfNotu() fonksiyonu neden public olmalıdır?

getHarfNotu() fonksiyonu, ortalama verisini kullanarak dışarıya anlamlı bir bilgi sunar. Eğer private olsaydı main() içinden çağrılmazdı ve yazBilgi() gibi başka public fonksiyonlardan çağrılabilmesi için her zaman sınıf içinde kullanılan bir yardımcı fonksiyon olarak kalırdı. Burada kullanım amacı dışarıya bilgi sunmak olduğundan public olmalıdır. Genel kural: dışarıya hizmet eden fonksiyonlar public, yalnızca iç işler için kullanılanlar private olmalıdır.

Gorev 5

Araç Kiralama Sistemi

Onceki konuları bir araya getirerek basit bir Araç Kiralama Sistemi geliştirin.

Sistemde her araç bir nesne olarak temsil edilir. Her aracın kendine ait bilgileri vardır:

- Plaka
- Marka
- Model
- Yıl
- Günlük kiralama fiyatı
- Kiralık olup olmadığı bilgisi

Bu bilgiler private olarak saklanır. Böylece aracın durumu doğrudan dışarıdan değiştirilemez. Program 3 farklı araç oluşturacak.

Daha sonra:

1. Birinci araç 3 günlüğüne kiralanacak
2. İkinci araç 7 günlüğüne kiralanacak
3. Birinci araç tekrar kiralanmaya çalışılacak, ancak sistem hata verecek
4. Birinci araç iade edilecek
5. Aynı araç tekrar 5 günlüğüne kiralanabilecek
6. Son olarak tüm araçların güncel durumu ekrana yazdırılacak

Bu sınıfta soyutlama nerede uygulanmaktadır?

kiralik değişkeni public olsaydı ne olurdu?

kirala() void yerine bool döndürecek şekilde nasıl tasarlanırdı?

Bu sınıfta soyutlama nerede uygulanmaktadır?

kirala() fonksiyonu çağrıldığında kullanıcı sadece araç plakasını, gün sayısını ve fiyatın hesaplanacağı değişkeni bilmek zorundadır. Aracın kiralık bayrağının nasıl değiştirildiği, toplam ücretin nasıl hesaplandığı ve durumun nasıl yönetildiği tamamen sınıf içinde gizlidir. Aynı şekilde bilgiYazdir() çağrısı tek satırla tüm bilgileri sunar. Kullanıcı iç detayları görmeden aracı kiralayabilir ve iade edebilir.

kiralık değişkeni public olsaydı ne olurdu?

Dışarıdan a1.kiralık = false; yazarak kiralık bir araç sahte olarak müsait gösterilebilirdi. Bu veri tutarsızlığına yol açar: araç gerçekte kirada olmasına rağmen sisteme müsait görünür. Kapsülleme sayesinde kiralık değeri yalnızca kirala() ve iade() fonksiyonları aracılığıyla değiştirilir. Bu kontrol olmadan sistemin güvenilirliği tamamen ortadan kalkar.

kirala() void yerine bool döndürecek şekilde nasıl tasarlanırdı?

bool kirala(int gun, double& toplamUcret) şeklinde tanımlanarak kiralama başarılıysa true, araç zaten kiralıksa false döndürülür. Bu sayede çağırın kod if (a1.kirala(3, ucret)) { ... } else { ... } yapısı ile sonucu kontrol edebilir. Mevcut cevap kodunda bu yaklaşım zaten uygulanmıştır. Alternatif olarak toplam ücreti dönüş değeri olarak verip hata durumunda -1 döndürmek de geçerli bir tasarımıdır.

Gorev 6

this İşaretçisi

this işaretçisinin neden kullanıldığını, sınıf içindeki fonksiyonların kendi nesnesine nasıl eriştiğini ve zincirleme metot çağrılarının (method chaining) nasıl çalıştığını inceleyin.

Normalde bir nesne üzerinde fonksiyon çağırırken:

```
s.artir();
```

yalnızca tek bir işlem yapılır. Ancak bazı tasarımlarda işlemleri arka arkaya daha kısa ve okunabilir şekilde yazmak isteriz:

```
s.artir().artir().azalt().yazdir();
```

Bu kullanım özellikle modern kütüphanelerde, ayar sistemlerinde ve nesne yapılandırmalarında çok yaygındır.

this->deger ne anlama gelir?

return *this ne döndürür?

Zincir çağrısının adım adım sonucu nedir?

this->deger ne anlama gelir?

this, üye fonksiyon içinde otomatik tanımlı bir işaretçidir ve 'bu nesne' anlamına gelir. Yani hangi nesne üzerinden fonksiyon çağrıldıysa o nesneye işaret eder. this->deger ise o nesnenin deger üye değişkenine erişmek demektir. Parametre ile üye aynı isimde olduğunda çakışmayı önlemek için kullanılır.

return *this ne döndürür?

*this ifadesi işaretçinin gösterdiği nesnenin kendisidir. Referans olarak döndürüldüğü için (Sayac&) kopyalama yapılmaz, nesnenin kendisi döndürülür. Bu sayede fonksiyon çağrıları zincirlenebilir: bir fonksiyon bittiğinde aynı nesne üzerinden hemen bir sonraki fonksiyon çağrılabilir.

Zincir çağrısının adım adım sonucu:

Başlangıç: deger = 0 (tanımsız, varsayalım 0). artir() -> deger = 1. artir() -> deger = 2. artir() -> deger = 3. azalt() -> deger = 2. yazdir() -> ekrana 2 yazdırılır. Sonuç: 2